

Zomato Lite

How a real product actually works — in one small app

What you will build: two screens, two APIs, two database tables, and one live URL on the internet.

What you will actually learn: where logic lives, and why that matters.

Who this is for: people with no coding experience. Nothing here assumes you have written a line of code before.

1 The one restaurant we care about

Ludhiana Burrito — Indian, Ludhiana-style burritos, Sector 32.

People eat there. People leave reviews. Our app lets them do exactly two things:

1. Write a review.
2. See the restaurant page with all the reviews.

That is the whole product. It is small on purpose — the ideas inside it are the same ones running Zomato, Swiggy and Amazon.

2 The five words, using a real restaurant

Forget code for two minutes. Imagine an actual restaurant.

Word	The restaurant version	What it really is
Frontend	The dining area. Menu, tables, the waiter you talk to.	What the user sees and taps.
Backend	The kitchen. Nobody eats there. All the work happens there.	Where rules and logic live.
API	The waiter. Takes your order in, brings food out.	The agreed way frontend talks to backend.

Word	The restaurant version	What it really is
Database	The cold storage. Raw ingredients, nothing cooked.	Where facts are stored permanently.
Git	The recipe book, with every edit dated and signed.	The history of every change to your code.
Vercel	Opening the restaurant to the public.	Putting your app on a real URL.

One rule explains most of this: the dining area never cooks. If the frontend is doing maths, something has gone wrong.

3 The database — and the thing most people get wrong

Two tables. That is it.

Table: restaurants

id	name	cuisine	area
1	Ludhiana Burrito	Indian	Sector 32

Table: reviews

id	restaurant_id	rating	comment	created_at
101	1	5	Paneer burrito is unreal	12 Aug, 7:14pm
102	1	4	Good, but slow service	14 Aug, 1:02pm
103	1	4	Solid. Would repeat.	20 Aug, 9:30pm

Now look at what is not in these tables:

- There is no `average_rating` column.
- There is no `latest_review` column.

Why not? Because they are not facts. They are results.

A fact is "Priya gave 4 stars on 14 August." That happened. Store it forever.

An average is just those facts, added up and divided, at the moment somebody asks. If you stored `average_rating` as a column, you would have to remember to update it on every single new review — and the day you forget, your app is lying to users.

The core lesson: store facts, compute answers.

The backend runs this every time the page loads:

```
averageRating  -> AVG(rating)           where restaurant_id = 1
totalReviews   -> COUNT(*)              where restaurant_id = 1
latestReview   -> ORDER BY created_at DESC LIMIT 1
```

Three lines. That is the entire intelligence of this product.

4 The two screens

Screen 1 — Write a review `/review/1`

- A star picker, 1 to 5
- A comment box
- A Submit button

Nothing else. When you press Submit, the frontend hands three things to the backend and steps back:

```
{ "restaurantId": 1, "rating": 4, "comment": "Good, but slow service" }
```

Screen 2 — Restaurant page `/restaurant/1`

Top to bottom:

1. Ludhiana Burrito — Indian, Sector 32
2. 4.3 stars — big, the first thing your eye lands on
3. The latest review, highlighted — because recency is what people trust
4. Older reviews, listed below

The frontend receives 4.3 and prints 4.3. It does not add. It does not divide. It does not sort. It renders.

5 The two APIs

An API is a contract: "send me exactly this, and I will send you back exactly that."

POST `/api/reviews` — save a review

In:

```
{ "restaurantId": 1, "rating": 4, "comment": "Good, but slow service" }
```

Out:

```
{ "success": true, "reviewId": 102 }
```

But first, the backend checks three things:

The check	If it fails
Is rating a whole number between 1 and 5?	Reject — 400

The check	If it fails
Does restaurant 1 actually exist?	Reject — 400
Is comment non-empty?	Reject — 400

GET /api/restaurants/1 — load the page

Out:

```
{
  "name": "Ludhiana Burrito",
  "cuisine": "Indian",
  "area": "Sector 32",
  "averageRating": 4.3,      <- computed: AVG(rating)
  "totalReviews": 3,        <- computed: COUNT(*)
  "latestReview": { ... },  <- computed: newest first, take one
  "reviews": [ ... ]
}
```

Notice that the response is shaped like the screen: name, big rating, latest review, then the list. Good API design means the frontend has nothing left to figure out.

6 Why the backend does not trust the frontend

A reasonable question: the star picker only shows 1 to 5, so why check for it again in the backend?

Because anyone can talk to your API directly. The frontend is just one polite visitor. A person with a terminal can send this without ever opening your app:

```
{ "restaurantId": 1, "rating": 500, "comment": "" }
```

No stars were clicked. No button was pressed. If the backend does not check, that 500 goes into your database, and Ludhiana Burrito now has an average rating of 169.

Frontend validation is kindness — it stops honest users making honest mistakes. **Backend validation is security** — it stops everyone else. Never remove the second one because you added the first.

7 One review, end to end

Trace a single review through the whole system. This is the trip that makes everything click.

1. Priya taps four stars and types "Good, but slow service."
2. The frontend packages it and sends it to POST /api/reviews.
3. The backend checks: rating in range, restaurant exists, comment not empty. All pass.
4. The backend writes one row into the reviews table. Nothing else in the database changes.
5. The backend replies: success, reviewId 102.
6. The frontend shows "Thanks!" and sends her to /restaurant/1.

7. That page calls GET /api/restaurants/1.
8. The backend runs AVG, COUNT and the sort — right now, fresh.
9. The frontend prints the numbers it was handed.

The average changed. But nobody ever updated an average. That is the point.

8 Git — the undo button that never expires

Git records every change you make to your code, with a note about why.

Four words you will use all day:

- **commit** — save a snapshot, with a message such as "added rating validation".
- **push** — send your snapshots to GitHub, so they exist somewhere other than your laptop.
- **branch** — a parallel copy where you can try something risky.
- **revert** — go back to a version that worked.

Why you should care as a PM: when an engineer says "let me check when this broke", Git is what they are checking. Every line of code has a name and a date attached to it.

9 Vercel — going live

Right now your app runs at localhost:3000, which only exists on your machine. Nobody else on earth can open it.

Vercel connects to your GitHub repository and does this:

```
you push to GitHub -> Vercel notices -> builds your app -> live URL
```

That is it. No servers to buy, no configuration to fight with. Push code, get a URL, share it with a friend in another city.

You will also learn a small painful lesson here. The app that worked perfectly on your laptop will break on Vercel the first time — usually a database connection or a missing environment variable. Everyone hits this. It is part of the curriculum, not a failure.

10 How the session runs

Block	What you do	What you walk away understanding
Hour 1	Set up the project, create the two tables, add Ludhiana Burrito by hand.	What a database row is.
Hour 2	Build both API endpoints. Test them with no user interface at all.	Backends exist independently of screens.

Block	What you do	What you walk away understanding
Hour 3	Build the two screens. Wire them to the APIs.	The frontend is a display layer.
Hour 4	Push to GitHub, deploy on Vercel, break it, fix it.	How software reaches real users.

You will deliberately try to break your own API with a rating of 500. Watching your backend reject it is more satisfying than it sounds.

11 Glossary

Term	Plain English
Endpoint	One specific address the backend answers on, such as <code>/api/reviews</code> .
GET	"Give me something." Reading. Changes nothing.
POST	"Here, take this." Writing. Changes the database.
JSON	The text format frontend and backend use to pass data around.
200	Worked.
400	You sent me something invalid.
500	I broke — my fault, not yours.
Schema	The shape of your tables: what columns exist, what type each one holds.
Deploy	Move code from your laptop to the internet.
Repo	A project folder that Git is tracking.

12 The four things worth remembering

- 1. Store facts, compute answers.** Ratings are facts. Averages are answers.
- 2. The kitchen cooks, the dining room serves.** Logic lives in the backend.
- 3. Validate twice.** Frontend for kindness, backend for safety.
- 4. Shape your API like your screen.** If the frontend has to do maths, the API design is wrong.

Every product you will ever spec is a bigger version of this.

Prompt

The prompt that builds the whole application

How to use this

3. Make an empty folder — Desktop/zomato-lite is fine.
4. Open a terminal inside that folder and start OpenCode.
5. Copy everything below the line marked THE PROMPT, paste it in, press enter.
6. Read what it says. It will stop and wait for you at every step — that is by design, not an error.

One caution when copying out of Word: if any quote marks turn curly (" instead of "), the commands will fail. Paste into a plain text editor first if you see them.

THE PROMPT — COPY FROM HERE DOWN

Who you are talking to

I have never written code. I am a product manager learning how a real application fits together. You are not just building this for me — you are teaching me while you build it.

Golden rules — follow these for the entire session

- **Explain before you do.** Before creating or editing any file, tell me in two or three plain sentences what that file is for and why it needs to exist. No jargon without a definition the first time you use it.
- **Stop at every milestone.** After each milestone below, stop. Print what you did, tell me exactly what to click or check, and wait for me to say "next". Do not run ahead.
- **Stop when I need to leave the terminal.** Creating accounts, copying connection strings, clicking Deploy — you cannot do these. When you hit one, print numbered steps and wait.
- **Build only what is specified.** No extra features, no extra pages, no "while I was in there I also added". If you think something is missing, say so and let me decide.

- **Never store a computed value.** There is no `average_rating` column and no `latest_review` column. Ever. If you catch yourself wanting one, stop and tell me why you wanted it — that is the most important lesson in this session.
- **The frontend does zero maths.** It receives 4.3 and prints 4.3. No adding, no dividing, no sorting in React.
- **Commit at every milestone** with a message a non-engineer could understand.

Stack — use exactly this, do not substitute

5. Next.js 15, App Router, TypeScript
6. Tailwind CSS
7. Neon Postgres, queried with `@neondatabase/serverless`
8. **Raw SQL only.** No Prisma, no Drizzle, no ORM. I need to see `AVG(rating)` with my own eyes.
9. Deployed on Vercel
10. No auth, no state library, no component library

Mo Check my machine

Check that Node 20 or newer, npm, and git are installed. Print the versions. If anything is missing, give me the install command for my operating system and stop.

Then ask me to confirm I have accounts on GitHub, Neon (neon.tech) and Vercel — all free, no card needed. Wait until I say yes.

M1 Scaffold the project

Create a Next.js app called `zomato-lite` in the current directory, with TypeScript, Tailwind, App Router and the `app/` directory. Skip `src/`, skip Turbopack prompts, skip the import alias question — use defaults.

Delete the default homepage boilerplate. Replace it with a single line of text so I can see the app is alive.

Run the dev server and tell me to open `localhost:3000`. Explain what "running locally" means and why nobody else can see it yet.

Stop.

M2 The database

First, stop and give me these instructions:

10. Go to neon.tech and create a free project.
11. Copy the connection string that starts with `postgresql://`
12. Paste it into this chat.

When I paste it, create a .env.local file with DATABASE_URL= and that value. Add .env.local to .gitignore if it is not already there, and explain to me in plain English why a secret must never go into Git.

Then create a file db/schema.sql with exactly this shape:

```
CREATE TABLE restaurants (
  id          SERIAL PRIMARY KEY,
  name        TEXT NOT NULL,
  cuisine     TEXT NOT NULL,
  area        TEXT NOT NULL
);

CREATE TABLE reviews (
  id          SERIAL PRIMARY KEY,
  restaurant_id INTEGER NOT NULL REFERENCES restaurants(id),
  rating      INTEGER NOT NULL CHECK (rating BETWEEN 1 AND 5),
  comment     TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Seed one restaurant — Ludhiana Burrito, Indian, Sector 32 — and three reviews for it, with created_at values a few days apart so "latest" is meaningful:

rating	comment	created_at
5	Paneer burrito is unreal	8 days ago
4	Good, but slow service	6 days ago
4	Solid. Would repeat.	2 days ago

Write a script I can run with npm run db:setup that applies the schema and the seed. Run it. Then show me the rows it created.

Point out to me, explicitly, the two columns that do not exist in these tables and why.

Stop.

M3 The two APIs, with no user interface at all

Build both endpoints. Do not build any screens yet. I want to see that a backend is a real thing that exists on its own.

POST /api/reviews

Accepts:

```
{ "restaurantId": 1, "rating": 4, "comment": "Good, but slow service" }
```

Validates, in this order, and returns HTTP 400 with { "error": "<plain English reason>" } on the first failure:

11. rating is an integer from 1 to 5
12. comment is a non-empty string after trimming whitespace
13. restaurantId refers to a restaurant that actually exists (this one is a database lookup)

On success, inserts exactly one row and returns HTTP 201:

```
{ "success": true, "reviewId": 104 }
```

GET /api/restaurants/[id]

Returns HTTP 404 if the restaurant does not exist. Otherwise returns exactly this shape:

```
{
  "name": "Ludhiana Burrito",
  "cuisine": "Indian",
  "area": "Sector 32",
  "averageRating": 4.3,
  "totalReviews": 3,
  "latestReview": { "id": 103, "rating": 4, "comment": "...", "createdAt": "..." },
  "reviews": [ { "id": 102, "rating": 4, "comment": "...", "createdAt": "..." } ]
}
```

Rules for this endpoint:

14. `averageRating` comes from `AVG(rating)`, rounded to one decimal place in the backend.
15. `totalReviews` comes from `COUNT(*)`.
16. `latestReview` is the newest review by `created_at`.
17. `reviews` is every review except the latest one, newest first — because the page shows the latest one separately and I do not want it printed twice.
18. If there are no reviews at all: `averageRating` is null, `totalReviews` is 0, `latestReview` is null, `reviews` is []. Say out loud how you decided this, because "what happens when there is nothing yet" is a question I will have to answer on every product I ever build.

Then test it with curl, in front of me

Run each of these and show me the output:

```
curl localhost:3000/api/restaurants/1

curl localhost:3000/api/restaurants/999

curl -X POST localhost:3000/api/reviews \
-H "Content-Type: application/json" \
-d '{"restaurantId":1,"rating":5,"comment":"Great"}'

curl -X POST localhost:3000/api/reviews \
-H "Content-Type: application/json" \
-d '{"restaurantId":1,"rating":500,"comment":"Great"}'

curl -X POST localhost:3000/api/reviews \
-H "Content-Type: application/json" \
-d '{"restaurantId":1,"rating":4,"comment":""}'

curl -X POST localhost:3000/api/reviews \
-H "Content-Type: application/json" \
-d '{"restaurantId":999,"rating":4,"comment":"Great"}'
```

Then call `GET /api/restaurants/1` one more time and show me that `averageRating` and `totalReviews` changed without anyone updating an average anywhere.

Stop.

M4 The two screens

Design direction

Restrained and quiet. No gradients, no emoji, no drop shadows, no purple. Off-white background, near-black text, one warm accent used sparingly. Generous whitespace. One column, mobile-first, content capped around 560px and centred. Rounded corners, but subtle. It should look like something a designer left alone on purpose, not something a template generated.

Screen 1 — `/review/[restaurantId]`

19. The restaurant name at the top, so I know what I am reviewing
20. A star picker, 1 to 5, clickable, with a clear selected state
21. A comment box
22. A Submit button, disabled until a rating is picked and the comment is non-empty
23. On submit: call `POST /api/reviews`, then send me to `/restaurant/[id]`
24. If the API returns a 400, show the error message from the response. Do not invent your own message — display what the backend said.

Screen 2 — `/restaurant/[id]`

Top to bottom, in this order:

1. Restaurant name, with cuisine and area underneath in smaller muted text
2. **The average rating, large** — the biggest thing on the page — with the review count next to it in small text
3. The latest review, visually highlighted so it clearly sits apart from the rest
4. The older reviews below, in a plain list
5. A link to `/review/[id]`
6. If there are no reviews yet, an empty state that invites the first one

This page calls `GET /api/restaurants/[id]` and renders what it gets. Show me the line where you print the average and confirm out loud that there is no calculation anywhere in this file.

Stop.

M5 Git

Initialise Git if it is not already, and walk me through what is happening as you go. Explain commit, push, branch and revert in one sentence each, at the moment we first use them.

Confirm `.env.local` is not in the commit. Show me the proof.

Then stop and tell me how to create an empty repository on GitHub, and wait for me to paste the URL. Push to it.

Stop.

M6 Go live on Vercel

Give me numbered instructions to:

1. Import the GitHub repository into Vercel

2. Add DATABASE_URL as an environment variable in Vercel project settings — with the same value from my .env.local
3. Deploy

Warn me before I click Deploy that if I skip step 2, the build will succeed and the site will still break, and tell me what that error will look like. Then let me try it.

When it is live, ask me to open the URL on my phone.

Stop.

M7 Break it on purpose

Now that it is live, run this against the production URL, not localhost:

```
curl -X POST https://<my-vercel-url>/api/reviews \  
-H "Content-Type: application/json" \  
-d '{"restaurantId":1,"rating":500,"comment":"hacked"}'
```

Show me that it is rejected. Then explain, in plain English, why the star picker in the UI was never enough protection on its own.

When everything is done

Print a table of every file we created, with one plain-English sentence each about what it does and which of the five layers it belongs to — frontend, backend, database, config, or deployment.

Then ask me these three questions and check my answers:

1. Where does the average rating come from?
2. What would break if we had stored the average as a column?
3. Why does the backend re-check the rating when the UI already limits it to 1 to 5?

Things you must not do

25. No login, signup, or user accounts
26. No editing or deleting reviews
27. No photo uploads
28. No restaurant listing page or search
29. No ratings-breakdown chart
30. No ORM, no localStorage, no client-side state library
31. **No average_rating column, under any circumstances**

Vercel token:

<https://vercel.com/account/settings/tokens>

Github token:

<https://github.com/settings/personal-access-tokens>

Neon tech token:

<https://console.neon.tech/app/projects/holy-mouse-78611486/branches/br-little-mouse-b5b121j9?database=neondb>

Store this token in .env file